Conway

The case for Conway: what it is, why it's shaped this way, and what it buys you.

---

1. The gap

Most agentic-coding harnesses treat context as an afterthought and agents as flat. The MAST study, *"Why Do Multi-Agent LLM Systems Fail?"* (Cemri et al., UC Berkeley, 2025; arXiv:2503.13657), annotated 1,600+ execution traces across seven popular multi-agent frameworks and measured end-to-end failure rates of **41–87%**. The two largest categories:

- **Specification / system-design failures (41.8%)**: lost conversation history, step repetition, termination failures.
- **Inter-agent misalignment (36.9%)**: context resets, information withholding, task derailment.

Both describe the same root cause. Context flow between agents is opaque. You cannot see what a child knew, where its context came from, or why it drifted, so when it drifts there is nothing to correct against.

On the single-agent side, the context window gets treated as a bottomless buffer. When it fills, the harness answers with invisible truncation or automatic compaction. Routing across local and cloud backends is a black box: a request fails and you can't tell which model served it, why it was chosen, or why it broke. "Subagents" are one blurry feature with a partial-inheritance knob whose semantics nobody would call a primitive.

Conway's bet is that these are the same problem, and that context is the scarce resource a harness should be making visible and steerable.

---

2. What Conway is

Conway is a Rust agent harness for agentic coding. It runs LLM-driven agents that call tools, fork and spawn children, and route across multiple model backends, behind an explicit permission system, with durable session persistence and full context provenance.

It ships as one library, and three consumption modes are written against the same public API:

- **Interactive TUI**, the primary surface. `conway-cli`'s terminal shell: a single-column, copy-paste-friendly conversation stream, a live `/`-command palette, an on-demand agent-tree panel, ephemeral side-questions, and explicit fork/spawn/steer control over running children.
- **Embeddable Rust library**: the same harness as a single dependency. Fully async, event-streamed, no process boundary, for host applications (an IDE, a Tauri app, a service) that need inference and agent orchestration in-process.
- **One-shot (`-p` / `--print`)**: a scriptable Unix filter. Model output on stdout, diagnostics on stderr, `text|json|jsonl` output, stable exit codes, and fail-closed tool permissions (an empty allow-list denies every tool, since there is no operator to prompt).

No capability is trapped in one surface. The TUI is where Conway aims to lead and where the fastest dogfooding loop lives, but everything it does is reachable from the library facade, and one-shot is

a first-class consumer of the same API.

All three get the same extension surface. Hooks and a small plugin API are how you change what the harness does with context, permissions, tools, and routing, and they're the same interfaces the built-in tools are written against.

---

3. The thesis

Context is the scarce resource in agentic coding. Attention degrades well before the nominal window limit, and every record a child inherits is a record it has to spend against. Most harnesses answer this with a built-in compactor, a generic policy that decides what to forget on the user's behalf. Conway refuses that. The harness's job is to make context visible and steerable, and to spend it deliberately. The mechanisms will evolve; the objective is fixed.

Today that means organizing work as a tree of agents. A high-level agent holds a wide view of the problem at low fidelity. As you descend, each agent's accumulated context narrows onto a specific area of the codebase while keeping the big picture at diminishing granularity. Deeper agents end up carrying the larger contexts, sharply focused rather than broad. The hierarchy exists to put the right detail with the right agent, not to make children small.

The primitives that move work and context between agents are borrowed from Unix, where a parent spawns a child with an environment it chose, the child does one focused thing, and its output is a stream that flows back to the caller. Conway does the same with agents. Fork hands a child the parent's entire context; spawn hands it none. That's two sharp primitives instead of one overloaded "subagent" knob, and the operator or the model picks which one, controlling what a child receives and where its work product lands. Every context segment carries typed provenance, so you can inspect what an agent knew and where it came from. Routing is declarative and explainable end to end, so a failed request is attributable rather than mysterious. The inference provider is separable from the context surface: swap backends, local or cloud, Anthropic or OpenAI-compatible, without changing how you work with the harness.

Underneath all of this is a bet on minimalism. Opinions in the core aren't free: each one is a behavior an extension has to accommodate or work around, and they accumulate until extending the harness costs more than it should. Conway ships the primitives and leaves the policy to you. Opinions arrive as plugins and hooks the operator opts into, rather than as behavior the core hardcodes.

---

4. The distinctive mechanisms

4.1 Fork and spawn

Fork hands a child the forker's entire effective context at the fork point, as an immutable prefix. Nothing is copied, so forking is cheap enough to do freely. Siblings forked at the same point share that prefix, and because it's a literal byte-prefix, backends that do prefix caching reuse it. After the fork the two sessions are independent: prompting the parent never reaches the child, and vice versa.

Spawn hands a child nothing. Name an agent definition and the child gets that definition's system

prompt and tool set. Omit one and it inherits the spawning session's role and model routing, the same way a roleless fork inherits its forker's.

There is no partial-inheritance knob. The separation is what makes the two compose: tournaments, adversarial panels, parallel exploration, and the aggregate pattern (fork, then spawn N differently-prompted children, then collect their results) all fall out of two building blocks instead of one overloaded feature.

Steering is a separate channel. Parents steer children at turn boundaries, never mid-generation, and can soft- or hard-cancel them; children report progress and terminal results back. A parent's `await` on a child can never hang, because the supervisor synthesizes a terminal result on panic, budget exhaustion, or cancellation. Messaging is not context inheritance; the two are orthogonal.

4.2 Provenance and the visible agent tree

Every context segment carries typed provenance. For any agent you can inspect what its context contains and where each part came from. The `/agents` panel shows every live and finished agent, how it was created, and its place in the hierarchy, so checking what a child knew is a normal thing to do rather than a debugging expedition.

Out of the box Conway doesn't touch your context: no records dropped, no system prompt rewritten, no tool set narrowed. A host that wants to instrument context behavior does it through the `ContextHook` port: mask records, rewrite the system prompt, narrow tools, react to overflow. The core provides the extension point and the operator writes the policy.

4.3 Routing you can explain

When a request fails, you can tell which model served it, why the router picked that one, and which layer broke (`conway routes explain`). Adapters treat per-model differences in tool-calling reliability, streaming, and prompt-caching support as real, instead of flattening every backend into a lowest-common-denominator interface. Failover is health-aware: a slow-but-alive local server and a genuinely dead one are different states, and one transient blip doesn't take out your only configured endpoint.

Routing is content-agnostic by default. The router resolves a role to an ordered set of candidates on declared capabilities alone, and reads no request text to do it. Nothing in the default path decides where a turn goes based on what you said, and prompt-cache reuse stays an economics optimization and never correctness-bearing.

A role is an alias the caller asks for, and what picks the role sits above the router: the calling code, an agent definition, or a plugin that spawns with a role it chose. Routing on what's in a request is policy you can write there. Whether a summarization turn should go somewhere cheaper than a refactor depends on your workload and your budget, so Conway leaves that judgment to you and keeps the decision explainable once you've made it.

The default is the point, not a limitation to apologize for. A harness that guesses where your turns should go is one you have to fight when it guesses wrong. Conway would rather do the predictable thing and give you the controls, and the controls keep growing: expanding the API's reach over internals is ongoing work, so more of what the core does today becomes something you can drive yourself.

4.4 A durable, inspectable record

Sessions persist as an append-only JSONL log, one file per session, and in-memory state is a cache over that record. The discipline is persist-before-act. The record survives crashes, sessions resume, and any persisted session can be forked from at any point, transitively across multi-level fork chains. You can read it with ordinary tools (`conway sessions list | show | tree | export`). What the model sees is a view over the record, and changing that view is explicit and reversible. The record itself is the truth.

4.5 Predictability over cleverness

The harness fails loudly. The admission gate rejects an oversized request rather than silently truncating it. Result-contract validation retries once and then refuses, so you never get a quietly bad result. A caller-chosen session id that collides gets an error pointing you at `--resume`, not a silent overwrite. Prompt-cache reuse is never correctness-bearing.

Conway also errs hard toward being unopinionated. Behaviors another harness might bake in, like repeated-step detection, automatic context compaction, or tool-set narrowing, belong in plugins and hooks the operator opts into.

4.6 A small core, extensible by construction

The Rust plugin API is the extension interface: stable, semver-disciplined, and the same surface the built-ins use. The filesystem, shell, subagent, and reporting tools are all implemented on it, so a third-party tool can do anything a built-in can. Nothing in the core is privileged. The tool-facing types are serialization-ready, which leaves cheaper extension surfaces like subprocess hosts or WASM as a layered addition rather than an upheaval.

Adding tools is the shallow end of this. The interfaces that matter let you program the harness's behavior itself. `ContextHook` sees every assembled request before it routes, and can edit or drop segments, rewrite the system prompt, or narrow the tool set the model is told about; `on_overflow` gives it a second pass when the payload still doesn't fit. Hooks are async, so a policy can call a model of its own to decide. The permission gate is supplied by the consumer, so the rule a tool call has to satisfy is your code, and a denial can carry a reason the model reads and adapts to. Record masking drops a record out of future calls while leaving it in the log, and it's reversible.

All of that goes through the published API, at whatever granularity you need. A harness you can program beats one that guessed right about your workflow, and the gap widens the longer you use it.

Compaction is the obvious example. Other harnesses ship a built-in compactor, a generic policy that decides what to forget on your behalf. Conway has no compaction feature. When a context needs condensing, you write that policy yourself, in a plugin you control.

The harness's responsibility ends at the permission gate. Sandboxing, worktree isolation, and file-conflict prevention belong to an agent's own tools, not the core.

4.7 Security and reliability posture

- An agent handle cannot drive a session it doesn't belong to. Cross-session access is rejected outright.

- Permission gates come in allow-list, deny-all, and interactive-prompt forms, with a callback surface for the embedder. One-shot mode defaults fail-closed.
- The supervisor guarantees a terminal result on panic, budget exhaustion, or cancellation, so a parent waiting on a child is never left hanging.

---

5. The interactive surface, realized

5.1 `/agents`, the single agent surface

Every row shows how the agent was made: `fork @seq N` with the inherited fork point, `@<agent_def>` for a spawn with a named definition, `(inherit)` for a spawn that took the parent's role and model, `(ephemeral)` for a throwaway `/ask` fork. A `v` key cycles a draw-time visibility filter (active-only, all, finished-only) that never mutates the tree. Ephemeral forks are full tree citizens with their provenance attached. `/tree` is a hidden alias that renders the same nodes unfiltered as plain text.

5.2 `/ask`, a single-turn question with three exits

Asking forks an ephemeral child of the asker, runs one turn, and opens a modal over the answer. Closing it forces exactly one choice: **fork** to promote the child to a persistent session, **pull in** to merge the question and answer into the parent's transcript and purge the child, or **discard** to purge outright. There is no fourth way out; quitting with the modal open discards. You decide, every time, whether an answer enters the durable record, and a crashed process leaves only residue the next startup sweeps.

5.3 Natural language on `/fork` and `/spawn`

Free text after `/fork` or `/spawn` gets classified by a cheap model, and the result appears in a confirmation card before anything is created: `[enter]` to confirm, `[e]` to edit the classified prompt in the input line, `[esc]` to fall back to the raw text and your default recipe. Inference never silently chooses. The classifier's output is untrusted and validated before use, so a hallucinated agent definition is stripped, an invalid recipe degrades to verbatim passthrough, and a confused cheap model can't break the command. Explicit `@<agent_def>` syntax and bare invocations skip inference entirely.

5.4 `conway_ask`, the model-facing version

The model gets the same primitive. `conway_ask` runs a prompt in an ephemeral fork of the calling agent and returns the child's full reply text, not a truncated summary, so an orchestrator can draft context for a `conway_subagent` spawn while keeping that drafting out of its own context window. An optional `tools` argument narrows the child's tool set, e.g. `{"prompt": "summarize the diff", "tools": ["read"]}` for read-only inspection. `ask` is a composition of fork, not a third primitive.

---

6. Who it's for

- **The interactive power user** driving coding agents from a terminal, who wants the agent tree and context flow visible and steerable rather than hidden.
- **The automation engineer** who needs the same harness behind a clean one-shot CLI: streaming, structured output, stable exit codes, fail-closed permissions.
- **The host application** that needs inference and agent orchestration in-process, behind a single Rust dependency, with a flat ordered event stream a UI can render directly.
- **The extender** building tools or backends on a stable, semver-disciplined plugin contract that the built-ins themselves use.
- **The operator** who needs explainable routing, predictable failure, and a durable record they can inspect with ordinary tools.

---

7. The commitments

The mechanisms will change: automated context curation, richer steering, broader extension surfaces, editor reach through an Agent Client Protocol adapter as that protocol matures. These won't:

1. **Granular, composable agent primitives.** Fork and spawn, not one blurry subagent.
2. **Deliberate context economy.** Context is the scarce resource, treated visibly and steerably, with policy in hooks rather than hardcoded in the core.
3. **Explainable, content-agnostic routing.** Every response traceable to which model served it and why; no prompt content read by default, and content-aware policy left to the operator to add.
4. **A durable, inspectable record.** Persist before acting; the record is the truth.
5. **A small, extensible core.** Capabilities as plugins, built-ins unprivileged, the harness's responsibility ending at the permission gate.
6. **Interactive-first, every mode reachable.** The TUI leads, but no feature is trapped in one surface.
7. **Predictability over cleverness.** Predictable failure over silent truncation, infinite loops, or surprising cost.

Conway is licensed **AGPL-3.0-only**. Free to distribute and modify as long as source is provided; running a modified Conway as a network service requires making the modified source available to its users. That's a deliberate choice for an agent harness, and it means Conway isn't meant to be a permissively-licensed library dependency inside closed-source software.